

Strata52

ABC Windows: Field Measurement Automation

Bad data physically should not leave the job site.

The core problem isn't paper. It's that technicians don't always know exactly what a job requires before they start measuring, and gaps or errors in what they capture are only discovered after they've already left. I propose a guided, checklist-driven capture flow: the app tells the technician what this specific job needs, soft validates measurements and prompts for supporting details like photos, notes live at the point of capture, and gives the manager an exception queue instead of a blind check of every field.

To prove the approach rather than just describe it, I built a working prototype: technician capture app and manager dashboard, running end-to-end against a real backend.

Workflow: Before → After

Today:

```
Job assigned → Tech visits site → Measures on paper  
→ Transcribes form later → Manager reviews → Missing info?  
Yes → Clarification / second trip  
No → Sent to billing → Production + inventory + billing
```

Information is validated too late; Paper notes are unstructured and sometimes illegible; Techs rely on memory transcribing later; Missing measurements cause expensive second visits; Errors propagate downstream.

Proposed:

```
Job assigned → Dynamic checklist generated → Tech captures measurements, photos, notes  
→ Validated live, on-device → Exceptions resolved before leaving  
→ Manager reviews structured submission  
→ Approved, or clarification requested  
→ Sent to production, inventory, billing
```

```
States: Assigned → In Progress → Submitted → Needs Clarification (loops back to Submitted) →  
Approved → Production → Completed
```

If a job type has no configured measurement template, the system doesn't invent one. It routes for manual configuration rather than guessing.



[ADD DEMO VIDEO LINK HERE]

2–3 min walkthrough: tech submits a complete job → tries an incomplete one, blocked → enters an unusual measurement, flagged live → manager approves a clean one → billing queue updates → one clarification round-trip.



[ADD REPO LINK HERE]

Runs against mocked job data and a mocked downstream queue, not live ABC systems — see Tech Stack below for what's a prototype shortcut vs. what a real deployment needs.

Validation Model

Not all validation treated the same way:

TYPE	EXAMPLE	BEHAVIOR
Hard-required	Missing width, missing window type	Blocks submission, cannot be missing
Warn + confirm	Width entered as 5"	Flagged live as the tech types; corrected, or explicitly confirmed (override)
Nudge, not block	No photo or notes	One-time, non-blocking prompt before submit, forcing uploads mid-visit isn't worth the friction

A technician can also flag any window "not confident" routing it into the manager's exception queue directly (for the cases no algorithm catches.)

Tech Stack

Prototype stack: optimized for speed and demonstrating behavior. **Production stack:** selected after discovering ABC's identity, device environment, ERP, accounting, connectivity, security, and integration constraints.

What doesn't change regardless of which client this is: A shared backend is not optional, a client-only app can't make the tech→manager→billing handoff exist at all. Concretely: mobile-first PWA for capture (no app-store step, offline-capable); Same app, desktop-responsive, for the manager; A backend API owning the source of truth; a real relational database (the prototype's flat JSON files are a demo shortcut, not a production plan); Object storage for photos; and an API/webhook layer into ABC's existing ERP, an integration point, not a replacement.

Vendor choices (cloud, database, auth provider) should follow ABC's existing stack. Absent that information, i propose the default is Node/Express + React + Postgres. boring, well-supported, not a locked-in recommendation.

MVP vs. MVP+

MVP: job list, guided per-window measurements, dynamic required fields, unit and plausible-range validation, optional photographs with a non-blocking nudge, manager review screen, clarification request, approval and structured downstream handoff, timestamped audit trail.

MVP+ (explicitly deferred):

- **Offline sync:** job sites often have poor signal; a well-understood technical pattern, not dependent on ABC's specifics.
- **Autonomous approval:** skip the manager click for flag-free submissions: a risk-tolerance and policy decision for ABC, not a technical one.

Job Intake & Where AI Helps

Everything above starts from "job assigned", but how does a job get there? Today, that's manual: someone takes a call or an order and types customer, address, and job details into whatever system ABC already runs. This prototype doesn't touch that step `mockJobs.json` stands in for it. But the same principle driving the rest of this proposal applies here too: intake shouldn't be the one unstructured step in an otherwise structured pipeline. Job creation should follow its own guided checklist: customer, address, job type, special requirements etc .. Window type changing the required measurement checklist is the same idea one level down: not built here, but the architecture already supports it (job metadata already flows into the form), so it's an extension, not a redesign.

That's also, deliberately, where AI belongs in this proposal, and nowhere else. The problems named in the case (forgotten fields, illegible notes, second trips, manual re-entry) are a workflow and software design problem, not an AI problem, structured capture and live validation solve them without any AI at all. Intake is different: it's the one step here that starts from unstructured human input, which is exactly the kind of translation task AI is actually good at.

Concretely:

- **AI-assisted intake:** extract structured job metadata (customer, address, job type) from an unstructured order: a call transcript, an email, instead of someone re-typing it by hand, and auto-select the right measurement checklist based on job type.
- **Computer vision assisted measurement cross-check:** shown alongside manual entry as a second opinion, not a replacement for the tape measure. flagged if the two disagree. Needs a calibration reference in-frame to be trustworthy.
- **Smarter anomaly ranges over time:** today's plausible-range validation (6"-120", etc.) is a reasonable but static guess. Once real order history exists, those ranges could be learned per window type or region instead of hardcoded.
- **Voice-to-structured-notes:** lower priority, genuinely nice-to-have. converting a spoken field note into structured text.

- **AI-drafted clarification notes:** when a manager flags a submission, an LLM drafts a starting note from the flags themselves ("width looks unusually narrow for a Bay/Bow window — confirm?") for the manager to edit or send, instead of typing one from scratch every time. A more flexible answer to the "clarification specificity" question below than hardcoded presets would be.

None of this is built here, and that's deliberate, each needs either ABC's real intake process or real usage data to design well. Proposing it without that would be guessing dressed up as automation.

Adoption & Rollout

None of this works if technicians don't trust it. A crew that's measured windows with a tape and a notepad for years won't switch because the software is elegant. they'll switch because it's obviously faster and doesn't get them blamed for someone else's mistake.

- **Pilot with 2–3 technicians first:** not a full rollout, this validates the workflow against real job-site conditions (signal, glare on a phone screen, gloves) before scaling, and produces real baseline numbers for the metrics below instead of guessed ones.
- **Keep the UI boring on purpose:** minimal required fields, no jargon, nothing that needs a training manual to submit a job.
- **A fallback for when the app fails mid-visit:** No offline sync in the prototype yet (see MVP+), until it exists, pilot technicians need a plain paper backup, not a dead end.
- **The manager's role changes as much as the technician's:** from checking every field to reviewing exceptions. That's a bigger behavioral shift than it sounds like, and it's worth their opinion before day one, not after.

What I Decided vs. What I Left Open

Decided: manager approval is always required, flagged or not. Whether a clean submission could someday skip that step is ABC's call, not assumed here. No authentication in the prototype, and no push notifications either — a technician only sees a clarification request by opening the app and checking. Both are real gaps for a live rollout, named rather than hidden.

Left open, for discovery with ABC (ranked by how much the rest of this proposal depends on the answer):

- **Downstream system:** what receives the production/inventory/billing order today. Does it expose an API, or is it re-entered by hand?
- **Measurement fields:** what's required today, and does it differ by window type or job type?
- **Second-trip cost:** what's the current rate?
- **Devices & connectivity:** what do technicians carry, and what's typical site signal?
- **Override authority:** should tech, manager, or both be able to override a flagged measurement?
- **Clarification specificity:** should a request point to one specific field, or is a free-text note enough — and does it always mean a data fix, or sometimes an actual site revisit?

- **Auth & security:** what does ABC actually require for a field crew?

Success Metrics

Targets must be evaluated against baseline metrics: technician administrative time per job, manager review time per job, first-pass completeness rate, percentage of jobs requiring clarification, repeat site-visit rate, time from site visit to approval, manual data-entry time

Illustrative targets, clearly labeled as hypotheses:

- Reduce technician transcription from 20 minutes to under 2 minutes. the 10x this case asks for.
- Raise first-pass completeness above 95%
- Reduce approval latency from the following day to minutes
- Reduce measurement-related repeat visits by 70–80%

For the common case: no second trip needed, because bad data was caught at capture. The full cycle time from site visit to billing-ready collapses from same-day-or-later (transcription, manager review latency, billing re-entry) down to minutes. the exact multiple depends on ABC's current review turnaround.

Turning this into a dollar figure just needs ABC's real inputs — job volume, loaded technician/admin cost, current second-trip rate. The formula stays constant regardless of what those numbers turn out to be:

$$\begin{aligned} \text{ROI} = & (\text{second-trip hours avoided} \times \text{loaded technician cost}) \\ & + (\text{data-entry time avoided} \times \text{loaded admin cost}) \\ & - \text{implementation cost, amortized over the rollout period} \end{aligned}$$